

EE 439: Introduction to Machine Learning

Project Report: Data First, Models Second

Member 1 Rameen (2023-EE-3) & Member 2 Navaal Noshi (2023-EE-5)

Spring 2026

Selected Dataset: Fashion-MNIST

1 Phase 0: Exploratory Data Analysis

1.1 Data Partitioning [Rameen (2023-EE-3)| Navaal Noshi (2023-EE-5)]

The dataset was split using a simple method where we used the `train_test_split` function from `sklearn.model_selection` and set the test size to 0.2, which means 20% of the data is used for testing and the remaining 80% is used as the development set. The original dataset contained 70,000 images, which after splitting resulted in 56,000 images for the development set and 14,000 images for the test set. The basic idea for this is what we used a `random_state` of 42 so the data is divided the exact same way every single time. We only used the development set in Phase 0, while the test set was kept separate for final evaluation.

1.1.1 Illustrative Figure 1 [Member 1 Rameen (2023-EE-3)]

- Why is this informative?

Principal Component Analysis (PCA) reduces the dataset into 2 dimensions called PC1 and PC2. This figure is informative because it converts a very high-dimensional dataset into a simple 2D visualization that is easier to understand. Even though PCA only uses raw pixel values, the scatter plot still shows how different classes are distributed in feature space. The first two principal components capture the largest variations in the dataset which are usually related to image shape, brightness, and structure. The second PCA figure is also useful because it separates classes into two groups: classes that are well separated and classes that overlap, making it easier to identify which classes are easy to classify and which are difficult.

- What does it inform?

The PCA scatter plot shows that some Fashion-MNIST classes form clear clusters in the 2D space. Classes like Trouser, Sandal, Sneaker, Bag, and Ankle Boot are more separated because they have more unique shapes and structures. On the other hand, classes like T-shirt/Top,

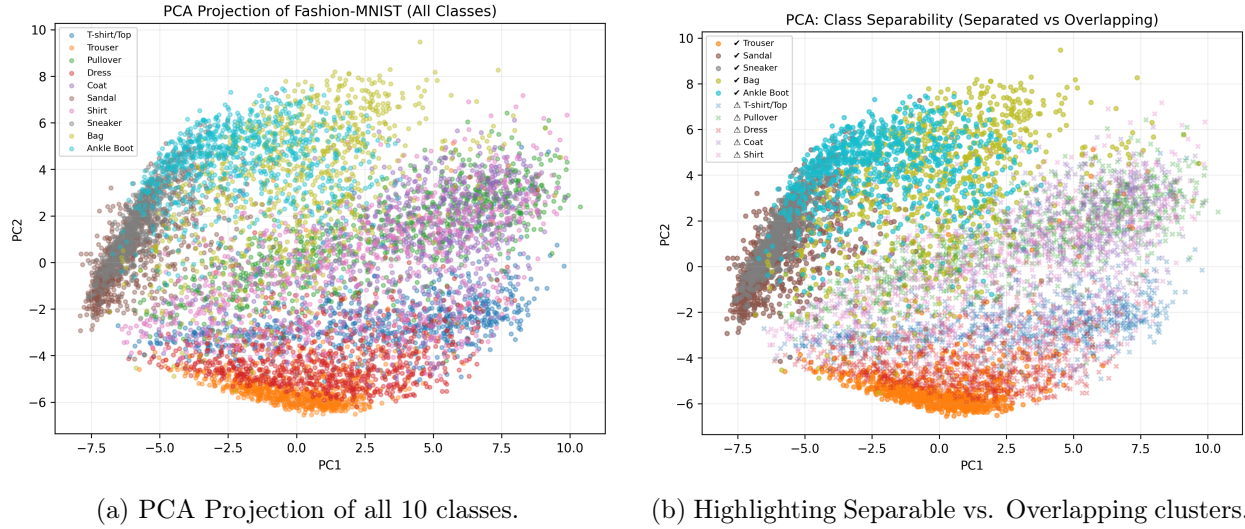


Figure 1: Comparative PCA analysis showing that while footwear and trousers form distinct clusters, upper-body garments (shirts, coats, pullovers) exhibit significant overlap.

Pullover, Dress, Coat, and Shirt overlap heavily in the center region because these classes have similar upper-body shapes. This tells us that the overlapping classes are more difficult to classify since they cannot be separated properly using simple linear projections.

- How does it shape your thought process?
This visualization helps us understand which classes will be difficult for the model to classify. Since upper-body garments overlap heavily, we can expect more confusion between these classes during training and testing. The PCA plot tells us that overall accuracy alone is not enough because some classes are naturally harder than others, so we should also check confusion matrices and per-class performance metrics. Overall, this figure helps guide preprocessing, model selection, and evaluation decisions.

1.1.2 Illustrative Figure 2 [Member 1 Rameen (2023-EE-3)]

- Why is this informative?
This figure is informative because it clearly shows different types of images in the training set in an organized way. It does not just display random data but arranges it into groups like typical, atypical, brightest, and darkest samples. This helps us understand how the data looks in both normal and extreme cases. It works like a test for the dataset showing whether it includes simple, clear images as well as difficult and unusual ones. Because of this, we can better judge if the training set is complete, balanced, and properly labeled.
- What does it inform?
This figure tells us several important things about the dataset. First, it shows that each class, like shirts, shoes, or bags, can have many different shapes and styles which means there is high variation inside one category. Second, it shows how lighting affects the images, as the brightest and darkest rows help us see if important details are still visible when the image is too light or too dark. Third, it shows that some classes can look similar in unusual cases which

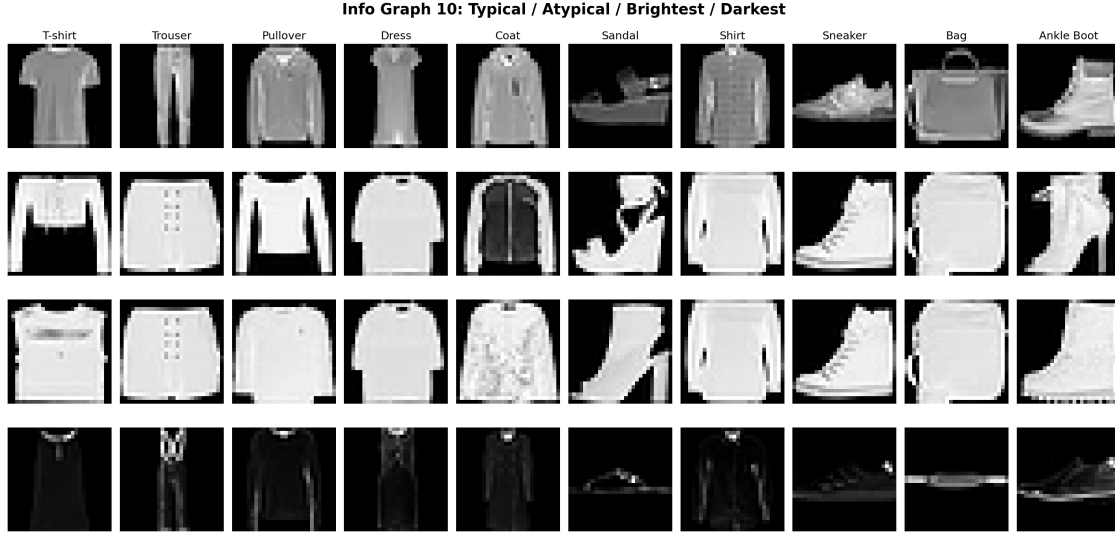


Figure 2: Grid showing Typical, Atypical, Brightest, and Darkest samples for each class, illustrating intra-class variance and exposure differences.

makes them harder to classify. Overall, it helps us understand how complex and diverse the data really is.

- How does it shape your thought process?

This figure changes how we think about building and improving a model. It makes us focus not only on normal images but also on difficult cases like very dark or unusual samples. It helps us realize that we may need to adjust the data, such as normalizing brightness, so the model learns better features. It also suggests adding more variety to the training set, for example by changing brightness or rotating images, so the model can handle different situations. Finally, it helps in debugging, because if the model makes a mistake, we can check if the image was too dark, too bright, or different from normal, and then improve the model accordingly.

1.1.3 Illustrative Figure 3 [Member 1 Rameen (2023-EE-3)]

- Why is this informative?

This visualization shows a 2×5 grid of histograms where each histogram represents the pixel intensity distribution of one Fashion-MNIST class. This figure is informative because pixel intensity distributions tell us important information about the images such as background area, object brightness, and contrast, and since all classes are shown together we can directly compare their intensity patterns and understand how visually different the classes are.

- What does it inform?

The histograms show that most Fashion-MNIST classes have a large spike near pixel value 0 because most of the image area is background but the remaining distribution changes from class to class. Classes like Trouser and Bag usually have higher mean pixel values because these objects cover a larger portion of the image and their histograms also spread more toward medium and high intensity values. On the other hand, classes like Sandal and Ankle Boot have smaller and more detailed structures so their pixel distributions are more concentrated

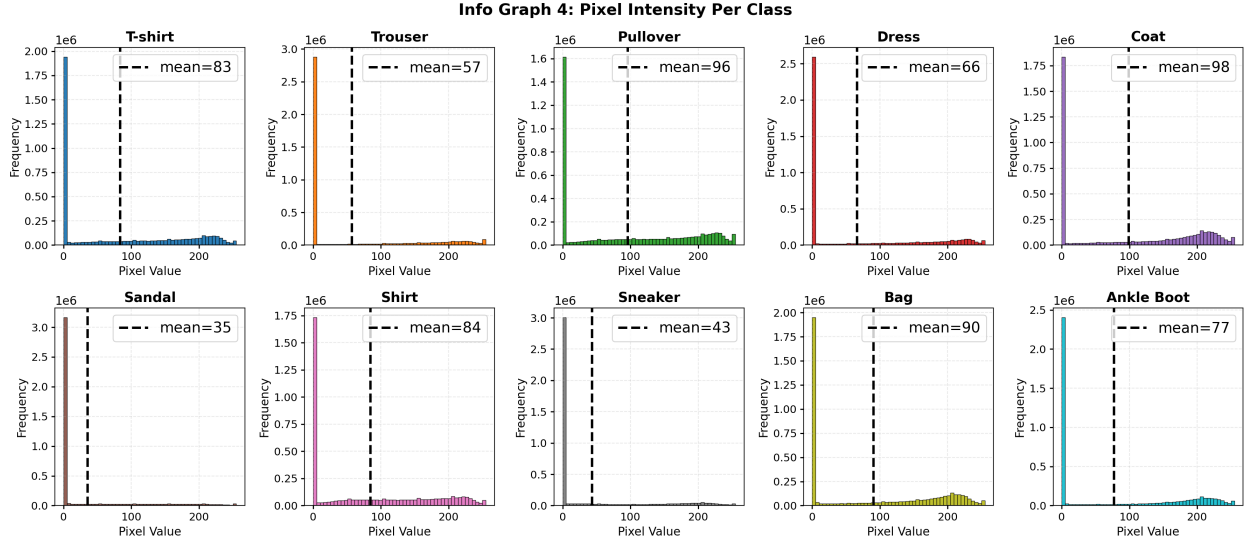


Figure 3: Histograms of pixel intensity distributions for all ten classes, with dashed lines indicating the mean intensity for each category.

around moderate intensity values. The figure also shows that some classes have multiple peaks, meaning images inside the same class can still look different in brightness and contrast, so overall pixel intensity is somewhat related to class identity but brightness alone cannot fully separate the classes.

- How does it shape your thought process?

This visualization helps us understand the importance of preprocessing before training the model because most pixel values are concentrated near zero and the distributions are highly skewed so normalization becomes important for better learning. It also shows that some classes are naturally brighter than others so the model may depend too much on brightness instead of learning actual object shapes which means augmentation techniques like brightness and contrast changes may be needed during training. Overall, this figure helps guide preprocessing, normalization, and feature extraction decisions before training the model.

1.1.4 Illustrative Figure 4 [Member 2 Navaal Noshi (2023-EE-5)]

- Why is this informative?

This graph is giving us important information, which is showing where exactly each class differs from all other classes. This is done by subtracting the mean of all other classes from one class and because of this we get a difference map. In this map only those pixels are highlighted which are important; meaning the ones which are uniquely bright or dark for that class and not present in others. So, this graph is more informative because it is not showing general appearance but it is focusing on the distinguishing pixels only which we cannot see in other graphs.

- What does it inform?

This clearly informs us about which pixels are actually playing a main role in distinguishing one class from another. It shows that the useful information is not present everywhere in the

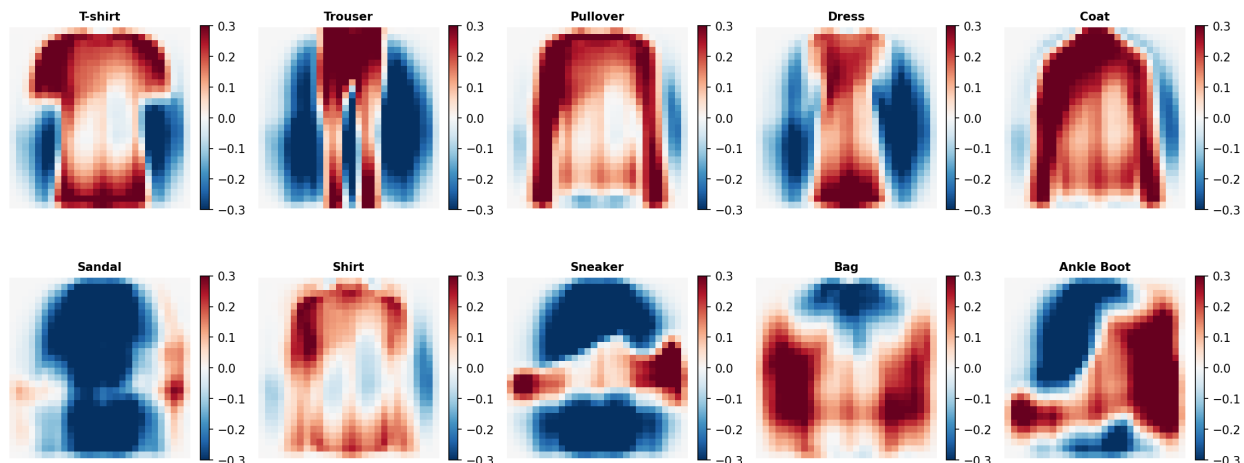


Figure 4: Difference map showing the uniquely bright pixels which are important for each class.

image, but it is only in specific areas. It shows us exactly what the model should look at to make the right choice.

- How does it shape your thought process?

This observation highlights that local structure is very important for this dataset. The basic idea for this is what standard methods treat every pixel as an independent piece of information, so because of this, they will fail to capture the actual underlying structure. The thing which needs to be done is to prioritize the techniques which are designed to look at local neighborhoods of pixels together. This helps us to clearly detect the shapes and structural patterns which are present in this data.

1.1.5 Illustrative Figure 5 [Member 2 Navaal Noshi (2023-EE-5)]

- Why is this informative?

This figure is giving a complete understanding of how similar or different all the classes are from each other before we even start the model training. The heatmap compares all the classes together, and the bar graphs highlight exactly those pairs which are the most confusing. So, this is more informative because it helps us to clearly identify which items will be easy to classify and which ones will create problems for the model.

- What does it inform?

Important thing in this figure is that the upper-body clothing classes like Shirt, Coat, and Pullover are highly similar and strongly overlapping with each other. The model may find it difficult to separate them correctly as their shapes and patterns look very similar with each other. But, classes like Bag, Sandal, and Sneaker are visually very different from the clothes, so they will be easier to handle. The ranked similarity graph is clearly showing the hardest pairs which are likely to produce the most errors.

- How does it shape your thought process?

The basic idea for this is what overall accuracy alone can be misleading, because the model can still confuse similar classes like Coat and Shirt. Model needs to learn fine local structural

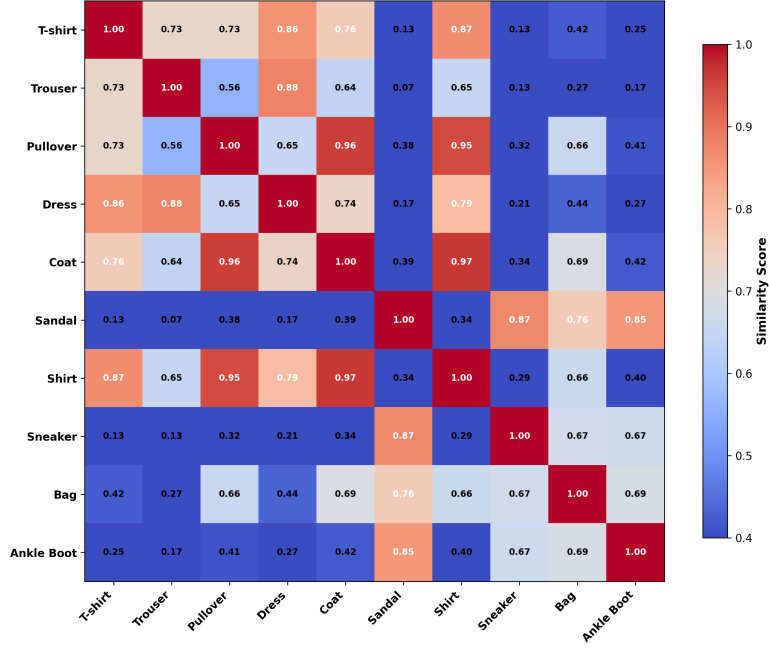
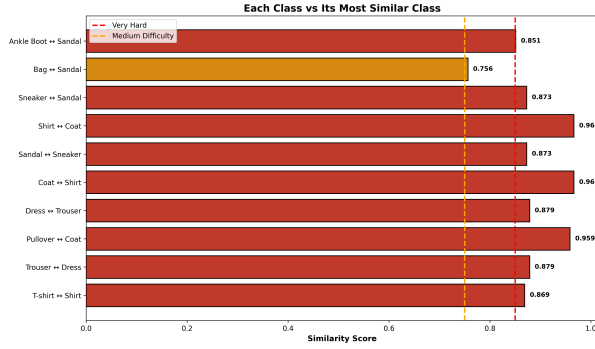
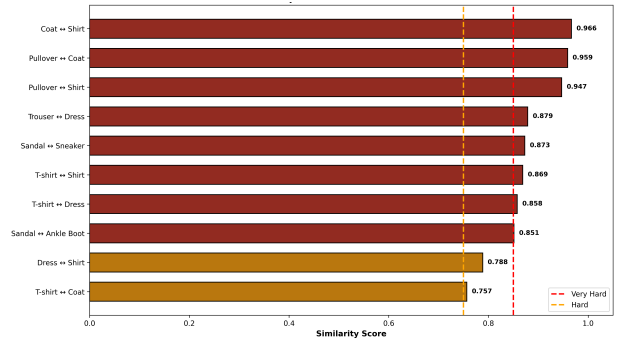


Figure 5: Visual similarity matrix between all classes. Higher similarity values indicate classes that are harder for the model to distinguish.



(a) Most similar class for each category based on average image correlation



(b) Top 10 hardest class pairs with the highest visual similarity scores.

Figure 6: Comparison of the most similar classes and the hardest class pairs based on visual similarity scores.

patterns, not just simple pixel values. The thing which needs to be done is to focus on confusion matrices to clearly see where the model is struggling.

1.1.6 Non-Illustrative Figure 1 [Member 2 Navaal Noshi (2023-EE-5)]

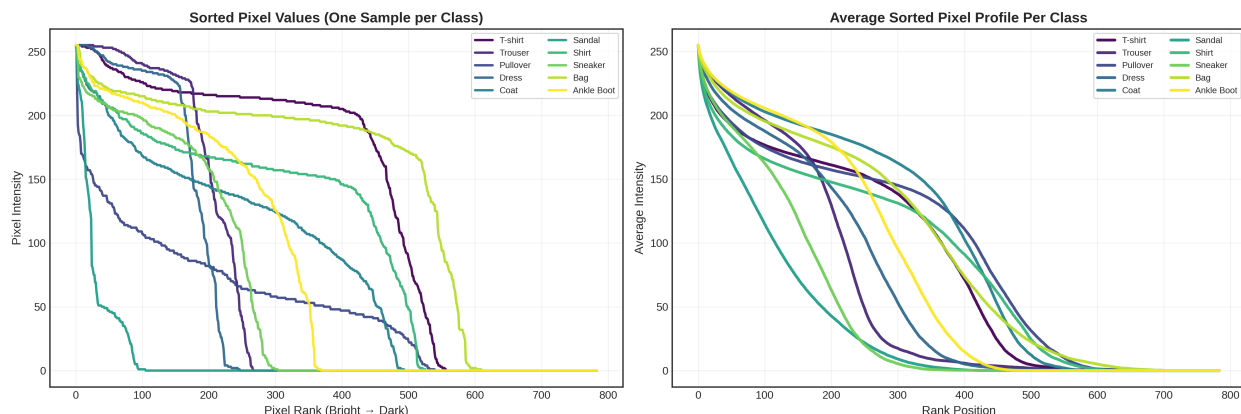


Figure 7: Sorting Pixels Destroys Shapes and Mixes Up the Classes

- Why is this non-informative?

This representation is non-informative because sorting the pixel values completely destroys the actual structure of the image. The basic idea for this is what even though the total number of bright and dark pixels remains the same, their exact position is completely lost. And position is exactly the thing which creates the shape of an object. Because of this, different classes like shirts and trousers start looking exactly the same, as they just have similar pixel values without any meaningful structure.

- How does it shape your thought process?

Real information in an image is not just the pixel values themselves, but how they are arranged together. It shows that keeping the local structure safe is very important for understanding the data. This clearly tells us that standard methods which treat every pixel as an independent value will fail here. We need to look for techniques which are specially designed to look at the spatial arrangement (local arrangement).

1.1.7 Non-Illustrative Figure 2 [Member 2 Navaal Noshi (2023-EE-5)]

- Why is this non-informative?

This representation is non-informative because Fashion-MNIST is purely a grayscale dataset. The basic idea for this is what each pixel has only a single intensity value representing brightness, no separate RGB content. If we try to analyze color, it will just produce identical results across R, G, and B because all three channels contain the exact same values. Applying a colormap like viridis does not introduce any real color information. It is only mapping the grayscale values to artificial colors just for visualization purposes. Because of this, these colors are not actually part of the dataset and do not reflect any real useful information.

- How does it shape your thought process?

As no color content is present (RGB), any techniques which are designed for color images analysis, color histograms will be completely useless for this dataset. This shows that the real

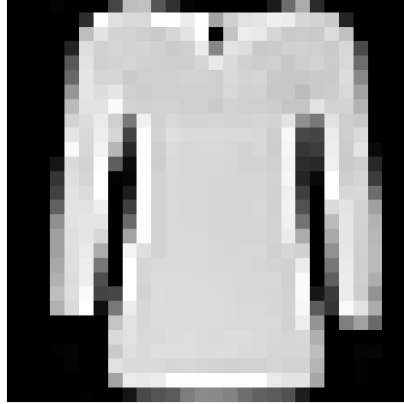
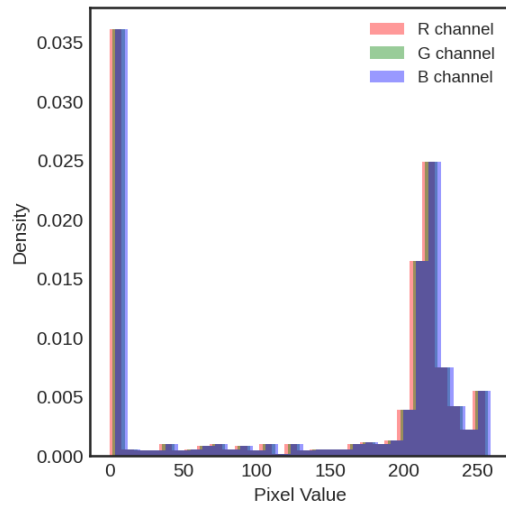
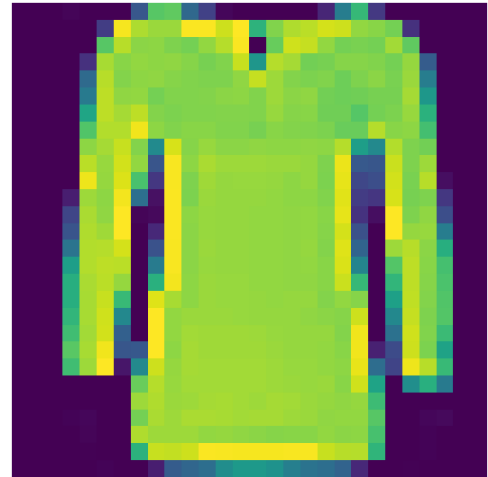


Figure 8: Correct grayscale visualization of a sample image from the data set



(a) RGB channels perfectly overlap, proving the data has no color information.



(b) Colors added by the Viridis map are just for display, not real features.

Figure 9: Demonstration of non-informative color analysis on grayscale data

information comes from the shape (arrangement) of the clothes, not the colors. We need to understand our data before working with it.

1.1.8 Non-Illustrative Figure 3 [Member 2 Navaal Noshi (2023-EE-5)]

- Why is this non-informative?

This graph is non-informative because looking at a 1D bar chart is just like looking at a random sequence of lines. The basic idea for this is what when we flatten a proper 2D image into a single straight line, all the visual patterns are destroyed. Because of this, it is impossible to tell if the chart is showing a shirt or a shoe, and we are just left with unrelated numbers.

- How does it shape your thought process?

One important thing is that keeping the pixels in their correct 2D layout is very necessary.

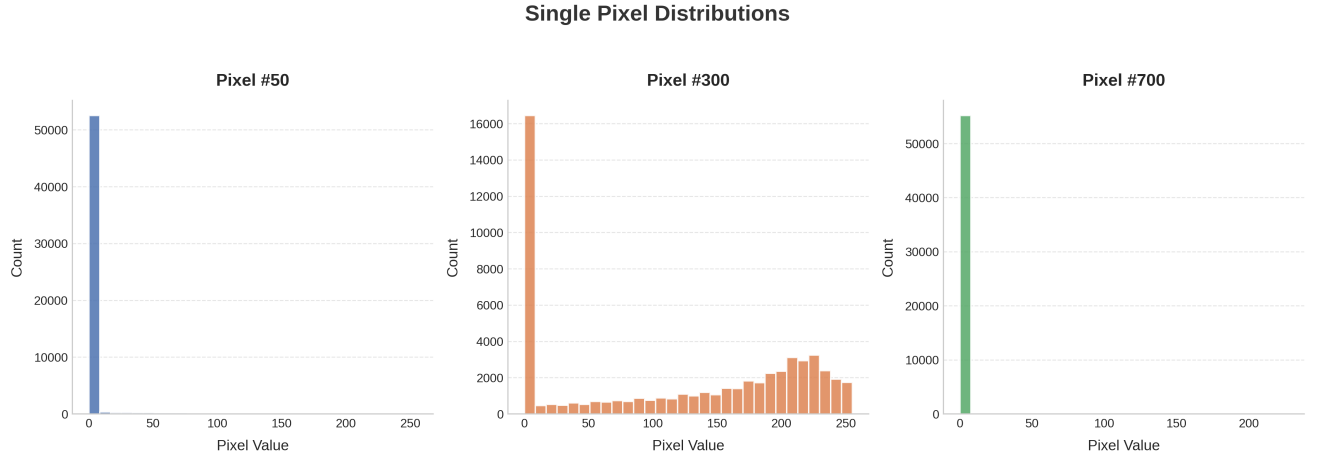


Figure 10: Frequency distribution of pixel intensities for three individual pixels, demonstrating varying levels of pixel sparsity

This tells us that standard methods which just flatten the image into a line will lose the actual shape and will completely fail on this dataset. The thing which needs to be done is to choose models which are designed to keep the pixel neighbors together so they can capture the shapes properly.

1.1.9 Non-Illustrative Figure 4 [Member 1 Rameen (2023-EE-3)]

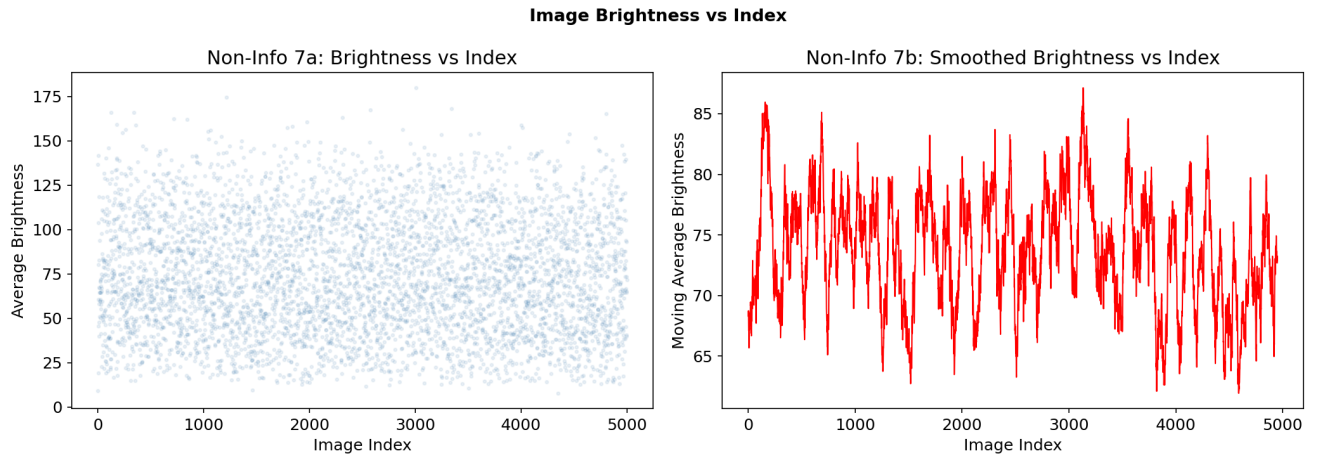


Figure 11: Scatter plot and moving average of average image brightness against the dataset row index

- Why is this non-informative?

This figure is non-informative because it plots average image brightness against the image index. The index has no real meaning in this dataset because the Fashion-MNIST images are not ordered in any logical sequence. So, no real pattern can exist along the x-axis. The scatter plot shows random spread with no trend which only reflects natural variation in image

brightness which is not a useful information. The moving average also stays flat with small random changes which again shows no meaningful pattern. Overall, this plot does not give any insight about classes or model learning because it uses a meaningless axis.

- How does it shape your thought process?

Even though this plot is not useful but it helps understand that brightness varies a lot between images which suggests that brightness should be normalized so it does not affect the model unfairly. It also shows that we should analyze brightness by class instead of by index. The smooth curve also confirms that the dataset is well shuffled, since there is no structure or sudden changes across the index. This gives confidence that the data is properly randomized and suitable for training a model.

1.1.10 Non-Illustrative Figure 5 [Member 1 Rameen (2023-EE-3)]

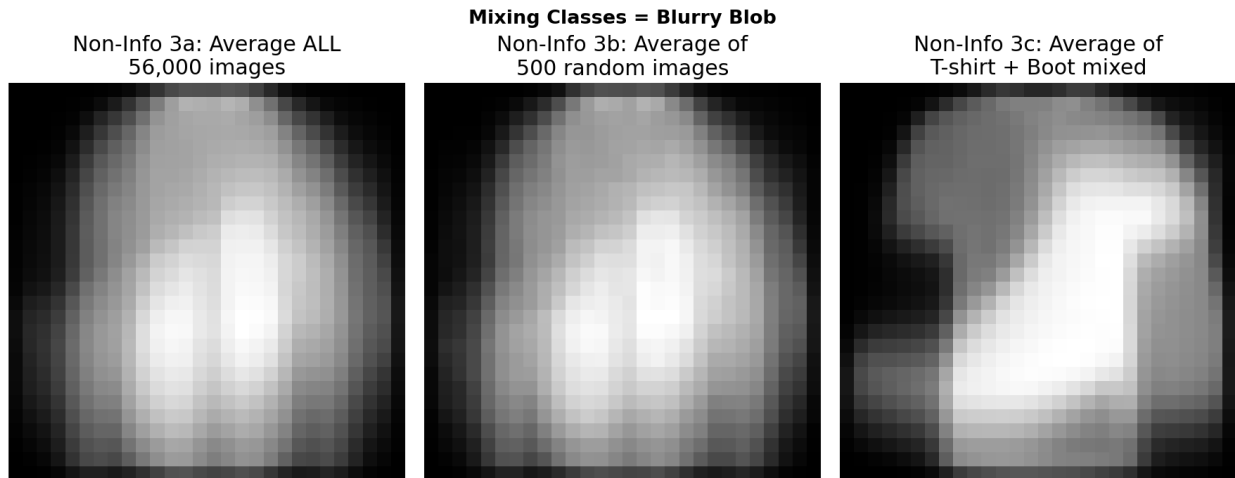


Figure 12: Mean image visualizations demonstrating the “Blurry Blob” effect caused by averaging randomized classes

- Why is this non-informative?

This figure is non-informative because it averages many images together which removes all clear shapes. When all classes are mixed, the result becomes a blurry gray image with no useful structure. The full dataset mean and the random sample mean look almost the same, so both only show an average blur, not real patterns. The mixed-class image (T-shirt and boot) also becomes a strange overlap that does not look like any real object. In all cases, important details like edges and class shapes are lost, so the figure does not help to understand the data.

- How does it shape your thought process?

This figure shows that mixing all classes is not useful because it hides important features. It tells us that we should calculate averages per class instead of all data together. It also shows that the dataset is well shuffled because the full mean and sample mean are very similar. The mixed image shows that different classes should not be combined because it creates noise. Overall, it suggests we should focus on class-wise patterns, not global pixel averages.

2 Phase 1: Hypothesis and Model Justification

2.1 Structural Observations

2.1.1 Class Separability and Overlapping [Rameen (2023-EE-3)]

The PCA plots shown in **Figure 1** from Phase 0 showed that Fashion-MNIST is not a simple linearly separable problem. PCA reduced the original 784-dimensional image space into only two dimensions (PC1 and PC2), but even after this reduction, some classes still formed visible clusters while others heavily overlapped. Classes like Trouser, Sandal, Sneaker, Bag, and Ankle Boot formed relatively clear and separated clusters because these classes have more unique shapes and structures. On the other hand, upper-body clothing classes like T-shirt/Top, Pullover, Dress, Coat, and Shirt overlapped heavily in the center region of the PCA plot because these classes have similar overall shapes. This overlap shows that their raw pixel patterns are very similar and difficult to separate using simple linear boundaries.

2.1.2 Intra-Class Variation and Brightness Sensitivity [Rameen (2023-EE-3)]

The representative image grid shown in **Figure 2** highlighted the intra-class variation and brightness sensitivity present in the Fashion-MNIST dataset. The Most Typical images showed the standard appearance of each class, while the Most Atypical images showed unusual samples that may cause classification errors because they look visually different from the normal class pattern. The Brightest and Darkest images showed that the same class can appear very different under different lighting and contrast conditions, which means brightness alone is not enough for classification. These observations showed the importance of normalization, augmentation, and learning shape-based features instead of relying only on pixel intensity.

2.1.3 Observations from Pixel Intensity Histograms [Rameen (2023-EE-3)]

The pixel intensity histograms shown in **Figure 3** also confirmed important properties of the dataset. Most classes had a very large spike near pixel value 0 which means that most of the image area is background and only a small portion contains the actual clothing object. Some classes like Trouser and Bag had slightly higher mean pixel values because these objects cover a larger area of the image, while classes like Sandal and Ankle Boot had more concentrated intensity distributions because they contain smaller and more detailed structures. The histograms also showed that some classes had multiple peaks which means images inside the same class can still vary in brightness and contrast. These observations showed that simple brightness information is not enough for classification and that the model must learn actual object shapes and structures.

2.1.4 Spatial Localization of Important Pixels [Navaal Noshi (2023-EE-5)]

From **Figure 4**, we can clearly see that the important pixels are not spread everywhere across the image. The basic idea for this is what the useful information is only located in very specific spatial

regions. For example, Trouser shows important pixels in two straight lines for the legs, Sandal shows them right at the bottom sole, and Bag forms a central rectangle. The empty background has no useful information at all. Because of this, treating every single pixel equally will just waste time. Also, Shirt and T-shirt show almost the exact same important areas. From this observation, one important thing is that just looking at the overall shape is not enough to separate them, so we must find very small, fine details inside those specific regions. This tells us that treating every pixel as an independent value will completely fail here. The thing which needs to be done is to find models which can specifically focus on these important spatial regions and look closely at the local neighbor pixels to know the correct detail.

2.1.5 Visual Similarity Between Classes [Navaal Noshi (2023-EE-5)]

From Figure 5, we can clearly see that some classes are much harder to separate than others. From this observation, one important thing is that upper-body clothes like T-shirt and Shirt are highly similar, making them the hardest pair to separate in the dataset. Bag is very different from everything else, making it the easiest class to identify. Just looking at overall accuracy can be very misleading. The basic idea for this is what the model can get a high overall score but still completely confuse similar classes like Coat and Pullover. The thing which needs to be done is to focus on confusion matrices so we can clearly see exactly where the model is struggling.

2.2 Hypothesis on Model Complexity

2.2.1 Limitations of Simple Models and Need for Non-Linear Learning [Rameen (2023-EE-3)]

A simple linear model such as Logistic Regression on raw pixels will not be enough for this problem. The PCA plots showed that many upper-body clothing classes heavily overlap in feature space which means the classes are not linearly separable using raw pixel values. The pixel intensity histograms also showed that most of the image area is background and that brightness alone cannot properly separate the classes. In addition, the representative image grid highlighted strong intra-class variation, where some samples within the same class looked very different because of lighting, contrast, or unusual shapes. These observations suggest that the model must learn local spatial patterns and object structures instead of only relying on global pixel intensity values. Because of this, models like CNNs or SVMs with non-linear kernels are expected to perform better than simple linear models. However, the model should also use proper regularization because the dataset contains many visually similar classes and an overly complex model may overfit by memorizing noise instead of learning general shape-based features.

2.2.2 The Risk of Overly Complex Models [Navaal Noshi (2023-EE-5)]

While simple models will fail here, using extremely complex models is also very risky. **From Figure 4**, useful information is only in specific spatial regions. The basic idea for this is what an overly complex model might learn fake patterns in the empty background, wasting power on useless pixels. **From Figure 7**, when spatial arrangement is removed, all classes look exactly the same.

Just making a model bigger will not help at all if it ignores this spatial structure. The thing which needs to be done is to use a balanced model that focuses on the pixel layout without memorizing useless noise. **From Figure 2**, we can see that some images are very confusing even for humans. From this observation, one important thing is that a very complex model will just memorize these confusing images instead of learning real patterns. Because of this, it will overfit and completely fail on new test data.

2.3 Model Selection

- **Baseline Model [Rameen (2023-EE-3)]:** Logistic Regression

Logistic Regression is one of the simplest multi-class classification models. It treats each pixel as a separate feature and tries to separate classes using linear decision boundaries. From the PCA plots, we already observed that many Fashion-MNIST classes overlap in feature space, especially the upper-body clothing classes, so the dataset is not fully linearly separable. Because of this, we expect the model to struggle on visually similar class pairs and achieve lower accuracy on difficult classes. However, using Logistic Regression as a baseline model is still important because it helps us compare how much improvement we get when we later use more advanced and complex models.

- **Main Model [Navaal Noshi (2023-EE-5)]:** Convolutional Neural Network (CNN).

*We choose the CNN as our main model. The basic idea for this is what it is the only model actually designed for 2D images. **From Figure 4**, we know the important pixels are only located in specific spatial regions. On the same note, the CNN looks at small patches of the image to catch exactly these local patterns. **From Figure 7**, we saw that destroying the spatial arrangement makes all classes look exactly the same. Because of this, models that flatten the image early will completely fail here. From this observation, one important thing is that the CNN never flattens the image, so it keeps the full 2D layout perfectly safe. **From Figure 5**, the hardest pairs like Coat and Shirt differ only in very fine details. The basic idea for this is what the CNN can easily catch these tiny differences by looking at edges and shapes step by step.*

- **Flexible Comparison [Rameen (2023-EE-3)]:** Support Vector Machine (SVM) with RBF Kernel.

An SVM with an RBF kernel is more flexible than Logistic Regression because it can learn non-linear decision boundaries. This is useful for Fashion-MNIST because many classes overlap in feature space and cannot be separated properly using simple linear boundaries. SVM works well on medium-sized datasets and can handle more complex patterns in the data. However, it still uses flattened pixel values and does not fully understand the spatial structure of images like a CNN. By comparing SVM with the other models, we can check whether a strong non-linear model on raw pixels is enough or if spatial feature learning is actually necessary for better performance.

2.4 Evidence Table

Evidence from Phase 0	Implication for Model Choice	Member
Figure 1 (PCA) shows T-shirt, Shirt, Coat, and Pullover overlap heavily with no linear boundary separating them.	A linear model cannot separate these classes. A nonlinear model such as a CNN is required.	Rameen (2023-EE-3)
Figure 2 (class distribution) confirms a perfectly balanced dataset with exactly 10% per class.	Standard accuracy is a fair metric. No oversampling or weighted loss functions are needed.	Rameen (2023-EE-3)
Figure 3 (pixel intensity histograms) shows the majority of pixels are zero-valued black background across all classes.	Normalization dividing by 255 is required. Models must not rely on absolute brightness values alone.	Rameen (2023-EE-3)
Figure 2 (atypical images) shows high intra-class variation with some samples ambiguous even to human observers.	The model must generalize, not memorize. Dropout regularization prevents overfitting to these hard ambiguous training samples.	Rameen (2023-EE-3)
From Figure 4, important pixels are only located in specific spatial regions.	CNN looks at small patches to catch exactly these patterns, but flat models will completely fail here.	Navaal Noshi (2023-EE-5)
From Figure 5 and Figure 6, Coat and Shirt are the hardest pair to separate, while Bag is the easiest class.	Overall accuracy alone is not enough. We must check confusion matrices. On the same note, a CNN is needed to tell these hard pairs apart.	Navaal Noshi (2023-EE-5)
From Figure 7, destroying the spatial arrangement makes all classes look exactly the same.	The exact 2D layout is the real information. The CNN keeps it perfectly safe, while flat models just destroy it immediately.	Navaal Noshi (2023-EE-5)
From Figure 9, we can clearly see the dataset has no real color. All three channels are exactly the same.	Using color techniques is completely useless. The basic idea for this is what all the useful information only comes from shapes and pixel patterns.	Navaal Noshi (2023-EE-5)
From Figure 10, flattening a 2D image into a 1D line makes it completely impossible to read.	Any model that flattens the image early will lose all shape information and completely fail here.	Navaal Noshi (2023-EE-5)

Table 1: Mapping Phase 0 data characteristics to model selection decisions for Fashion-MNIST.

3 Phase 2: Model Implementation and Evaluation

3.1 Model Implementation

3.1.1 Baseline Model [Member 2 Navaal Noshi (2023-EE-5)]

- Preprocessing

The first step for the implementation was the normalization of the pixel values. As it was highlighted in analysis, the pixel values were normalized by dividing by 255. The basic idea for this is what the model learns better and the gradient descent converges faster when all input values are on the same scale (between 0 and 1). Another important thing as mentioned is that each 28x28 image was then flattened into a 784 dimensional vector. I choose to do this because Logistic Regression requires a 1D input and cannot work with the 2D image structure directly. This relates back to Figure 7 and Figure 10, where it was showed that flattening the image destroys spatial arrangement, which is the disadvantage of we make when using a simpler baseline model

- Hyperparameters

For this model i wrote the code from scratch as per the instructions. The basic idea for this is what we started all the learning numbers (weights and bias) at exactly zero for all 10 classes. Then i set the learning speed to 0.1. From this observation, one important thing is that it controls exactly how big of a step the model takes to learn. The model practiced for 500 rounds for the main training (epoch), and on the same note, it practiced for 500 rounds just to check its work . I used a simple math rule to change the raw scores into clear percentages for all 10 classes. The thing which needs to be done is to train the model to learn from its mistakes. Because of this, in every single round, the model checks the difference between its guess and the real answer, and updates its numbers so it can make the right choice.

- Python library

The model as per instructions was built from scratch and the library used is only numpy.

3.1.2 Main Model [Member 1 Rameen (2023-EE-3)]

- Preprocessing

I normalized pixel values of Fashion-MNIST images by dividing them by 255 so that all values range from 0 to 1 instead of 0 to 255. This helps the CNN learn faster and more smoothly because large input values can make training unstable and slow. When the data is in a small and consistent range, the model updates its weights more evenly, gradients start to behave better and the network becomes more stable and accurate during training. After that, each image is 28×28 pixels in Fashion-MNIST which means it has 28 rows and 28 columns of numbers. Each image is gray, so it has only 1 channel (not RGB). Here, N means the number of images in the dataset. Instead of making the image flat (784 numbers), I change the shape from $(N, 784)$ to $(N, 28, 28, 1)$. I do this so the CNN can see the image in its real 2D form. The "1" means one color channel (gray image). This helps the CNN understand shapes and patterns better because it can use the real image structure.

- Hyperparameters

I built the CNN completely from scratch using only NumPy. The model has two convolution

layers where each layer uses 16 filters of size 3×3 to detect small patterns like edges and shapes. He initialization was used so that the values stay stable after ReLU activation and training does not explode or vanish. After the first convolution layer, a 2×2 max pooling layer was applied which helped to reduce the image size from 28×28 to 13×13 by keeping only the strongest features. The second convolution layer reduced it further to 11×11 to only focus on the important patterns. After that, I flattened the output into a single vector and its size became 1936 ($11 \times 11 \times 16$). This vector was then passed into a fully connected layer with 256 neurons and ReLU activation so that the model can learn more complex patterns. I also used a dropout layer with a rate of 0.4 to reduce overfitting by randomly turning off some neurons during training. The final output layer has 10 neurons with softmax activation to predict the 10 clothing classes. The model was trained using momentum SGD with a learning rate of 0.01, momentum of 0.9 and learning rate decay of 0.95 after each epoch. I used a batch size of 64 and trained for 40 epochs because the loss was still decreasing in earlier tests so more training helped to improve the performance.

- Python library

The whole model was built from scratch using only NumPy. Libraries like TensorFlow and PyTorch were not used. All parts of the CNN such as convolution, max pooling, dropout, backpropagation and momentum SGD were implemented manually.

3.1.3 Flexible Comparison [Member 2 Navaal Noshi (2023-EE-5)]:

- Preprocessing

For the SVM, I applied the same preprocessing as I did for Logistic Regression. The basic idea for this is what both models require the same input format to work correctly. The pixel values were normalized by dividing by 255 so they fall between 0 and 1. On the same note, each image was flattened into a 784-dimensional vector. The reason why I choose to do this is because SVM also requires 1D input and it does not understand the 2D spatial structure directly. As I mentioned in my earlier tasks, this means we are losing the shape information which is a big trade-off for this model.

- Hyperparameters

For the SVM model, I go through several settings to handle the complexity of the data. The first step was setting the kernel to RBF. The reason why I choose RBF is because it allows the model to learn curved (non-linear) boundaries. This is very important because, as I highlighted in my PCA plots, the classes overlap a lot and a simple straight line is not enough to separate them. Another important thing is the regularization parameter C , which I set to 10. The basic idea for this is what it allows the model to fit the training data more closely by being more strict with errors. On the same note, I set the gamma to scale. This is a good choice because it automatically changes based on our data and the number of features we have. For the multi-class part, I used the 'ovr' (One-Versus-Rest) strategy. This helps the model deal with the ten different types of clothes by comparing one class against all others at a time. Finally, the random state was set to 42 so that when running the code each time the result does not change.

- Python library

I have used the scikit-learn library for this task because it provides all the necessary tools for SVM and parameter tuning in a simple way.

3.2 Performance Metrics

3.2.1 Baseline and Flexible Model Metrics [Navaal Noshi (2023-EE-5)]

For the evaluation part, the Logistic Regression and SVM models were tested on the test set which has 14,000 images. This was done after training them on the development set of 56,000 images. The basic idea for this is that both models require the image to be flattened into a 784 dimensional vector. As mentioned before, this means they both lose the spatial structure, which was already highlighted as critical in Figure 4 and Figure 7.

Another thing is that the macro F1 score was the primary evaluation metric. This is especially important here because, as shown in Figure 5, overall accuracy alone can be misleading. This happens when hard class pairs like Coat and Shirt exist in the dataset, so a metric that clearly reflects performance across all classes is necessary.

Metric	Logistic Regression	SVM
Overall Accuracy	82.08%	90.33%
Macro F1 Score	0.8190	0.9027
CV Accuracy Mean	0.8246	0.9034
CV Accuracy Std	0.0049	0.0035
Coat F1	0.74	0.85
Shirt F1	0.56	0.73
Pullover F1	0.71	0.84

Table 2: Performance of Logistic Regression and SVM RBF on Fashion-MNIST test set. Per-class F1 scores for the three hardest classes confirm the predictions made in Phase 0.

Both models struggle the most on the upper-body clothing classes as we get idea from the per-class F1 scores in Table 2. Logistic Regression shows weaker performance overall because it can only draw linear boundaries between classes. SVM performs better since the RBF kernel allows for non linear boundaries but at the same time still it lost its spatial struture due to the image flattening.

3.2.2 Main Model Metrics [Rameen (2023-EE-3)]

The CNN model was tested on the same unseen test set of 14,000 images. Unlike Logistic Regression and SVM which converts each image into a flat 784-dimensional vector and lose spatial structure, the CNN kept the full 2D image layout throughout the forward pass. This allowed the model to learn from spatial patterns instead of treating pixels independently. This directly resolves the issue highlighted in Figure 4 and Figure 7, where important visual information is lost after flattening.

Metric	CNN
Overall Accuracy	92.08%
Macro F1 Score	0.9213
CV Accuracy Mean	0.9034
CV Accuracy Std	0.0038
Coat F1	0.88
Shirt F1	0.78
Pullover F1	0.88

Table 3: Performance of CNN on the Fashion-MNIST test set. Per-class F1 scores show strong improvement on difficult categories compared to flat models.

The CNN achieved an overall accuracy of 92.08% and a macro F1 score of 0.9213 which is higher than Logistic Regression (82.08%, F1: 0.8190) and SVM RBF (90.33%, F1: 0.9027). A key point is that this CNN was built completely from scratch using NumPy without libraries like TensorFlow or PyTorch. Even then, it still outperformed a tuned scikit-learn SVM by 1.75 percentage points in accuracy and 0.0186 in macro F1. This confirms that preserving spatial structure is more important than using a more complex classifier on flattened data.

The training curve shows smooth and stable learning. The loss decreased steadily from 0.5723 at epoch 1 to 0.0616 at epoch 40, without sudden jumps or instability. The learning rate was gradually reduced from 0.01 to 0.00129, which helped the model learn faster in early stages and fine-tune later. The consistent drop in loss indicates good convergence.

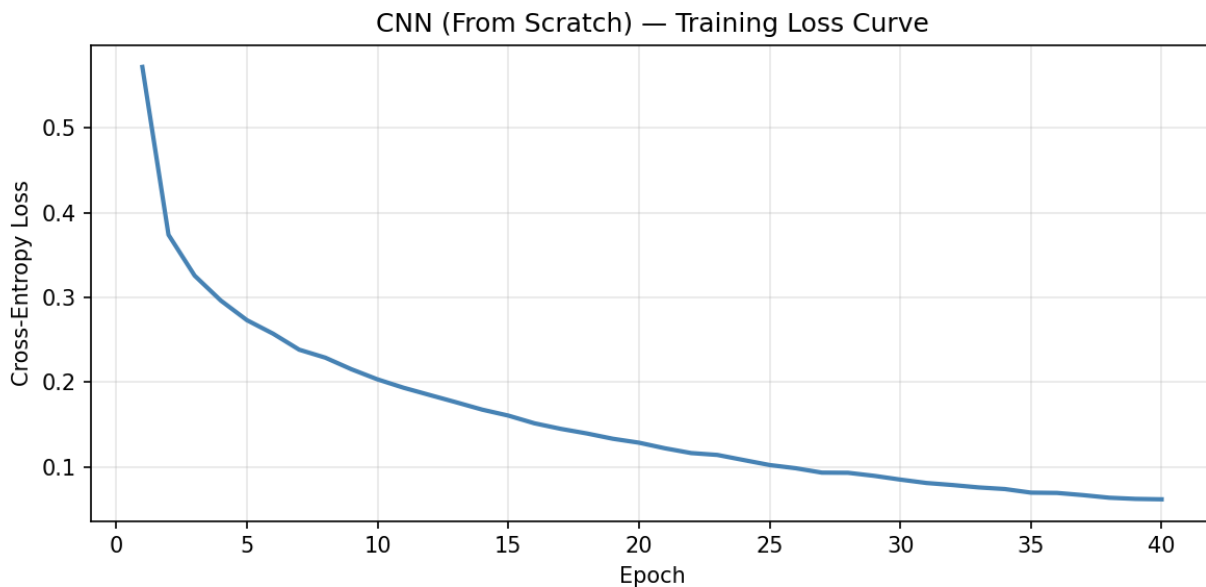


Figure 13: Training loss curve showing smooth convergence across 40 epochs.

The cross-validation results (0.9034 ± 0.0038) show that the model generalized well and did not overfit. Performance remained stable across different splits of the data.

The per-class results highlight the advantage of learning spatial features. The hardest classes identified earlier were Coat, Pullover, and Shirt. For these, the CNN achieved F1 scores of 0.88, 0.88, and 0.78 respectively. Compared to Logistic Regression (0.74, 0.71, 0.56) and SVM (0.85, 0.84, 0.73), the improvement is clear. This matches the earlier visual similarity findings in Figure 5, where these classes were expected to be difficult due to overlapping shapes.

In contrast, clearly distinct classes such as Trouser, Sandal, Bag, and Ankle Boot achieved very high performance, with F1 scores around 0.97 or higher. This is consistent with the PCA results in Figure 1, where these classes formed well-separated clusters due to their distinct shapes.

Despite improvements, Shirt remains the most challenging class. It is often confused with T-shirts due to very similar structure. However, the CNN still improves its F1 score to 0.78, showing a clear gain over both baseline models. The stable validation score (0.9034 ± 0.0038) confirms that the model learned general patterns instead of memorizing difficult cases.

3.3 Hypothesis Testing

- **Did the results support your Phase 1 hypothesis?** [Member 1 Rameen (2023-EE-3)]

The Phase 1 hypothesis is fully supported by the final results. All three predictions made earlier were confirmed in Phase 2 testing. This can be seen from the test accuracy, cross-validation scores, training loss curve, and the confusion matrix which clearly shows where the model performs well and where it makes errors.

Prediction 1: Logistic Regression on overlapping classes

In Phase 1, PCA in Figure 1 showed that T-shirt, Shirt, Coat, and Pullover overlap in feature space and are not linearly separable. Because of this, it was expected that Logistic Regression would perform poorly on these classes.

The results match this expectation. Logistic Regression achieved 82.08% accuracy, but its weakest class scores were Shirt (0.56), Coat (0.74), and Pullover (0.71). These are the lowest values across all classes. This confirms that a linear model cannot properly separate strongly overlapping pixel distributions.

Prediction 2: Advantage of CNN through spatial learning

The second prediction was that CNN would perform better because it keeps the 2D image structure instead of flattening it. This idea came from Figure 2, Figure 3, and Figure 4, which showed strong spatial variation in the dataset.

The results strongly support this. The CNN achieved 92.08% accuracy and a macro F1 score of 0.9213, which is higher than Logistic Regression and also better than SVM RBF (90.33%, F1: 0.9027). Even though SVM is non-linear, it still works on flattened inputs, which removes

spatial information. The CNN performs better because it learns edges, shapes, and local patterns using convolution filters. This improvement is especially clear in difficult classes like Coat (0.88), Pullover (0.88), and Shirt (0.78).

The training loss curve also supports this behaviour. The loss decreases smoothly from 0.5723 at epoch 1 to 0.0616 at epoch 40, showing stable learning without sudden jumps or instability.

Loss Curve Observation: The smooth downward trend shows that the model is learning gradually instead of memorising the training data.

Prediction 3: Shirt as the hardest class

The third prediction was that Shirt would remain the most difficult class. This was based on Figure 5, which showed that Shirt and T-shirt have very similar visual structure.

The final results confirm this clearly. Shirt has the lowest CNN F1 score (0.78) and is often confused with T-shirt. This happens because both classes share almost identical shapes, making them hard to distinguish even for a CNN.

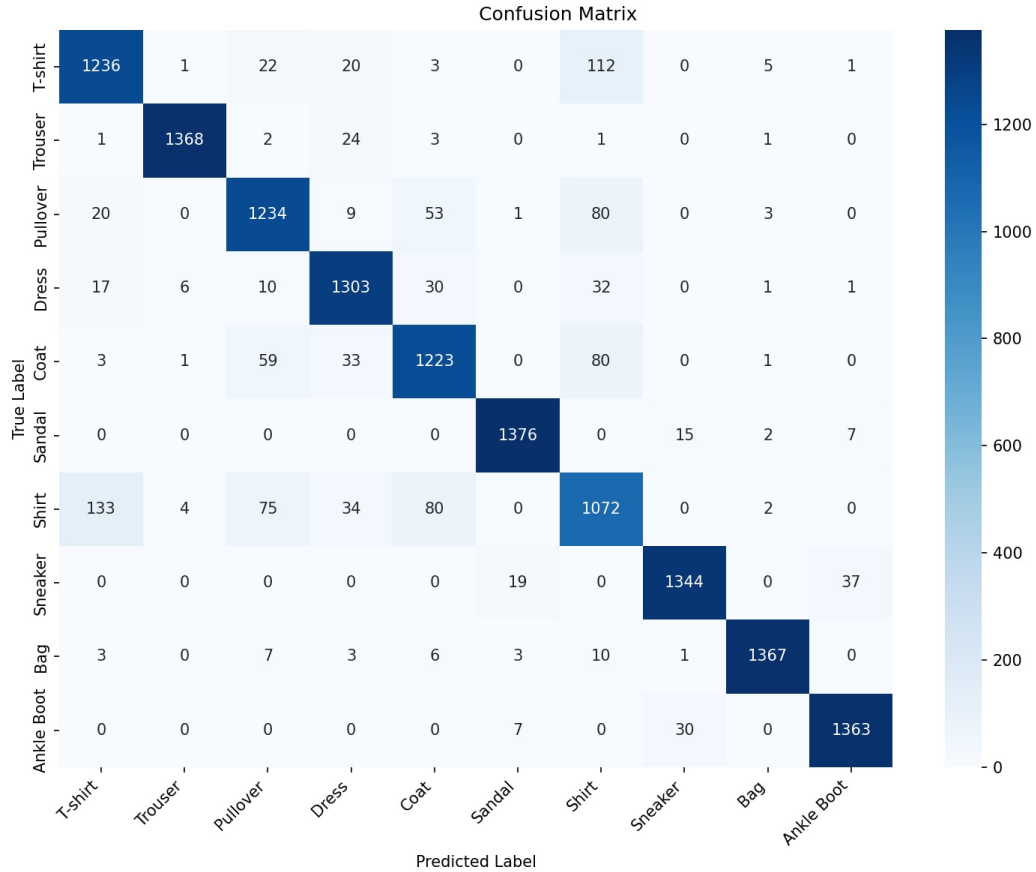


Figure 14: Confusion matrix of the CNN model showing correct predictions on the diagonal and misclassifications mainly between similar classes like Shirt and T-shirt.

The confusion matrix helps explain this better. It shows that Shirt is mainly misclassified as T-shirt and Coat, while T-shirt is also sometimes predicted as Shirt. These errors follow a

clear pattern based on visual similarity rather than random mistakes.

Confusion Matrix Observation: Easy classes like Trouser, Sandal, and Bag show strong diagonal values, meaning they are classified correctly. In contrast, Shirt, T-shirt, Coat, and Pullover show more off-diagonal errors due to their similar shapes.

- **Compare the performance of the Baseline vs. Main vs. Flexible models** [Member 2 Navaal Noshi (2023-EE-5)]

All three models show clearly different results. From this observation, one important thing is that it directly proves my Phase 1 hypothesis: any model ignoring the 2D layout (spatial structure) will underperform on Fashion-MNIST.

Logistic Regression got the lowest results with an accuracy of 82.08% and a macro F1 score of 0.8190. The basic idea for this is what we already guessed from Figure 1: simple straight lines just cannot separate classes that look very similar. Because of this, the model really struggled to make the right choice for the hardest clothing items. From this observation, one important thing is clearly shown in Table 2, where the most difficult classes got the lowest scores: Coat (F1: 0.74), Shirt (F1: 0.56), and Pullover (F1: 0.71).

SVM is our flexible model, and it was built using a very good built in library called scikit-learn. It achieved an accuracy of 90.33% and a macro F1 of 0.9027. The basic idea for this is what the RBF kernel helps the model to draw curved lines instead of straight ones. Because of this, it can handle the overlapping classes much better. But to use this model, the 2D image must still be flattened into a single straight line. From this observation, one important thing is that the real shape of the clothes is completely lost, just as we discussed in Figure 7. Coat improved to 0.85, Shirt to 0.73, and Pullover to 0.84.

The CNN was built completely from scratch without using any built in libraries. Because of this, getting an accuracy of 92.08% and a macro F1 of 0.9213. The overall accuracy is actually higher than SVM. The biggest advantage of the CNN is that it keeps the full 2D shape perfectly safe and looks at small patches to find patterns. From this observation, one important thing is that it directly solves the problem we saw in Figure 4. The CNN outperforms SVM on every hard class: Coat (F1: 0.88), Shirt (F1: 0.78), and Pullover (F1: 0.88).

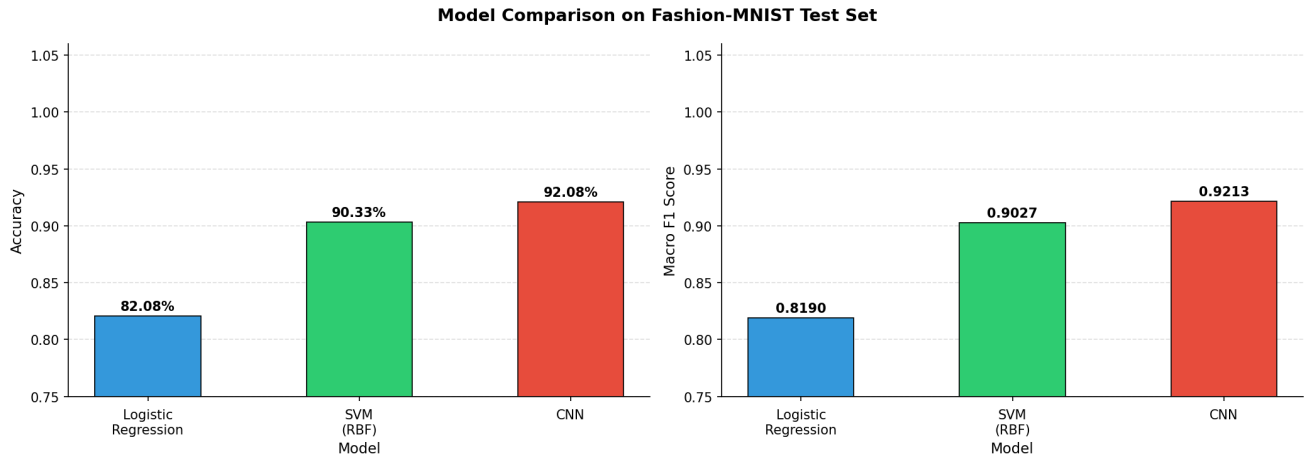


Figure 15: Overall accuracy and macro F1 score comparisons across Logistic Regression, SVM RBF, and CNN.



Figure 16: Per-class F1 scores for Coat, Shirt, and Pullover across all three models.

4 Phase 3: Reflection

1. Was your original hypothesis correct? [Navaal Noshi (2023-EE-5)]

The basic idea for this is what my Phase 1 hypothesis was correct. I predicted three things and here is what actually happened.

The first prediction was that Logistic Regression would perform the worst because it draws straight lines and Fashion-MNIST classes are not linearly separable. This was completely correct. Logistic Regression got 82.08% accuracy and struggled the most on Coat (F1: 0.74), Shirt (F1: 0.56), and Pullover (F1: 0.71). These were exactly the hard classes I identified from Figure 5 in Phase 0. On the same note, from Figure 1, we already saw that these upper-body classes overlap heavily in PCA space, so a linear model had no chance of separating them correctly. The confusion between these classes happened exactly how it was predicted.

The second prediction was that CNN would be the best model because it keeps the full 2D spatial layout. From Figure 4, I showed that useful information is only located in very specific spatial regions. On the same note, from Figure 4 itself, the empty background pixels carry no useful information at all, which directly showed that any model treating every pixel equally and ignoring spatial arrangement would completely fail. This prediction was also correct. CNN achieved 92.08% accuracy and outperformed both flat models on every single hard class pair. Coat improved to 0.88, Pullover to 0.88, and Shirt to 0.78. From this observation, one important thing is that as clearly shown in figure 16, the gap between CNN and the other two models is biggest exactly on these hard classes. This directly confirmed that spatial awareness is the real advantage of CNN over flat models.

The third prediction was about SVM. I predicted it would perform better than Logistic Regression but worse than CNN. The basic idea for this is what SVM uses curved boundaries which help with overlapping classes, but it still flattens the image. This prediction was surprising.

SVM achieved 90.33% which was close to CNN at 92.08%. The thing which needs to be done here is to note that SVM used the highly optimised scikit-learn library while CNN was built completely from scratch using only NumPy. From this observation, one important thing is that the CNN still performs better than SVM despite being built from scratch, which actually makes the result even stronger evidence for my hypothesis. As seen in figure 15, the CNN is clearly better in both accuracy and macro F1. A library-optimised CNN would likely outperform SVM by a much larger margin.

Overall my hypothesis was correct. The ordering I predicted was LR worst, CNN best, SVM in the middle and this is exactly what the results showed.

2. Which observations were most predictive of success? [Navaal Noshi (2023-EE-5)]

After going through the complete results, following observations from Phase 0 were the most predictive of success.

The first and most important observation was Figure 4, the pixel importance map. This graph showed that useful information is only located in very specific spatial regions. Trouser showed two vertical columns for the legs. Sandal showed a horizontal band at the sole. Bag showed a central rectangle. The basic idea for this is what this directly told me that any model ignoring spatial structure would fail. This single observation was the strongest justification for choosing CNN and it turned out to be completely correct. From this observation, one important thing is that as confirmed in figure 16, the CNN improvement on hard classes was the biggest exactly where this map highlighted the spatial differences between similar classes.

The second most important observation was Figure 5, the visual similarity matrix. This graph showed that Coat and Shirt have a similarity score of 0.966, the hardest pair in the entire dataset. On the same note, from Figure 6, the ranked bar chart made it even clearer by showing exactly which pairs would cause the most errors before training even started. From this observation, one important thing is that I could predict exactly which classes would cause the most errors before training even started. The results confirmed this perfectly. Shirt remained the hardest class for all three models. The ranking of hard classes in Phase 0 matched almost exactly with the ranking of lowest F1 scores in Phase 2. Because of this, the visual similarity analysis was a reliable predictor of model difficulty.

On the same note, Figure 4 was also very predictive for the flattening argument. This graph showed that the useful information is only in specific spatial regions and the empty background carries nothing useful at all. Because of this, I was confident that flat models would struggle since they treat every pixel equally without caring about position. The fact that Logistic Regression scored 82.08% while CNN scored 92.08%, a gap of 10 percentage points, directly confirms what this figure was telling us. The thing which needs to be done in future work is to use even more detailed spatial analysis in Phase 0 to predict not just which classes will be hard but by exactly how much.

3. Which observations were misleading? [Member 1 Rameen (2023-EE-3)]

After completing the project, I found that two observations from Phase 0 were somewhat misleading when compared with the final results from Phase 2.

The pixel intensity histograms suggested that image brightness might help distinguish some classes. In Figure 3, classes such as Trouser and Bag had higher average pixel

values than classes such as Sandal and Ankle Boot. This made me think that brightness could help separate some classes. However, the final results showed that brightness was not very useful for classification. The CNN focused more on shapes, edges, and spatial patterns instead of overall brightness. The confusion matrix showed that the model mostly confused classes with similar shapes, not classes with similar brightness. Therefore, the histogram analysis gave the impression that brightness was important when it actually was not.

The PCA plots made some classes look easier to separate than they really were. In Figure 1, classes such as Trouser, Bag, and Ankle Boot appeared clearly separated from the other classes. Because of this, I expected these classes to be classified almost perfectly. The CNN did achieve very high F1 scores for these classes, but PCA only shows a two-dimensional view of a much larger 784-dimensional feature space. A class that looks separated in a PCA plot may not be completely separated in the original data. Since PCA removes much of the information in the dataset, it should only be used as a rough visual tool and not as a strong indicator of classification difficulty.

These observations did not lead to a wrong model choice because the decision to use a CNN was supported by stronger evidence. However, they created slightly incorrect expectations about which features would be most important during training.

4. **Future approach: How would you pivot if you started over?** [Member 1 Rameen (2023-EE-3)]

If I started this project again, I would make three main changes to the exploratory analysis and the model design.

First, I would focus more on edges and texture patterns instead of brightness analysis. The histograms in Figure 3 and the brightness plot in Figure 11 required a lot of work but did not provide much useful information. Instead, I would use edge detection methods such as Sobel or Laplacian filters and compare the edge maps of different classes. This would show more clearly why classes such as Shirt, T-shirt, Coat, and Pullover are difficult to distinguish. It would provide a better understanding of the challenges in the dataset than simple brightness comparisons.

Second, I would improve the CNN architecture by adding a third convolutional layer and batch normalization. The current model uses two convolutional layers. Adding another layer would help the network learn more detailed and higher-level features. Batch normalization would make training more stable and allow the use of a larger learning rate. This could help the model reach better performance in fewer epochs. It might also improve the Shirt class, which had the lowest F1 score in the final results.

Third, I would use data augmentation during training. The image examples in Figure 2 showed that images within the same class can have different lighting conditions and orientations. However, the CNN was trained only on the original images. I would apply simple augmentation methods such as random horizontal flips and small brightness changes. This would help the model become more robust to variations in the data and improve its ability to classify unusual samples correctly.

Overall, I would still choose a CNN because it was clearly the right model for this dataset. The main changes would be to focus the exploratory analysis more on spatial features and to improve the model using a deeper architecture, batch normalization, and data augmentation instead of simply training for more epochs.

Statement of Contribution and AI Usage Disclosure

Rameen (2023-EE-3):

- Phase 0 Illustrative Figures 1, 2, and 3 (written analysis and bullet points);
- Phase 0 Non-Illustrative Figures 4 and 5 (written analysis and bullet points);
- Phase 1 Structural Observations Sections 2.1.1, 2.1.2, and 2.1.3: written analysis connecting Phase 0 figures to dataset properties;
- Phase 1 Hypothesis Section 2.2.1: argument for why simple linear models are insufficient and why spatial feature learning is needed;
- Phase 1 Model Selection: justification for Logistic Regression as baseline and SVM with RBF kernel as flexible comparison model;
- Phase 1 Evidence Table: rows 1, 2, 3, and 4;
- Phase 2 Main Model (CNN): full implementation from scratch using NumPy only, including conv layers, max pooling, dropout, fully connected layers, backpropagation, momentum SGD, learning rate decay, cross-validation, and all evaluation metrics;
- Phase 2 Performance Metrics: CNN results including overall accuracy, macro F1, cross-validation mean and standard deviation, and per-class F1 scores for all ten classes;
- Phase 2 Hypothesis Testing: full analysis of whether Phase 1 predictions were confirmed, including confusion matrix interpretation, training loss curve analysis, and comparison of CNN against both flat models;
- Phase 3 Reflection Question 3: analysis of which Phase 0 observations were misleading and what should be interpreted more carefully;
- Phase 3 Reflection Question 4: future approach discussion including augmentation, deeper architecture, and more targeted Phase 0 analysis for hard class pairs.

Navaal Noshi (2023-EE-5):

- Phase 0 Illustrative Figures 4 and 5 (analysis and bullet points);
- Phase 0 Non-Illustrative Figures 1, 2, and 3;
- Phase 1 Structural Observations Sections 2.1.4 and 2.1.5;
- Phase 1 Hypothesis Sections 2.2.2 and 2.2.3;
- Model Selection: Main Model (CNN) with full justification;
- Evidence Table rows 5, 6, 7, 8, and 9;

- Phase 2 Baseline Model (Logistic Regression from scratch using NumPy only): preprocessing, hyperparameters, and implementation;
- Phase 2 Flexible Comparison Model (SVM with RBF Kernel using scikit-learn): preprocessing, hyperparameters, and implementation;
- Phase 2 Performance Metrics: evaluation of Logistic Regression and SVM on test set including accuracy, macro F1, cross-validation, and confusion matrices;
- Phase 2 Hypothesis Testing: comparison of all three models and discussion of results against Phase 1 predictions.
- Phase 3: Was your original hypothesis correct?
- Phase 3: Which observations were most predictive of success?

AI Usage Disclosure [Rameen (2023-EE-3)]

PHASE 0: Figures and Analysis

Tool: Claude / ChatGPT

- **Clarification: PCA Scatter Plots:**
 - **Purpose:** After reading the Scikit-learn PCA documentation (<https://scikit-learn.org/stable/modules/decomposition.html>), I asked AI to confirm whether my understanding of PCA projection was correct.
 - **Impact:** AI helped by explaining my own understanding and ChatGPT added a lot more clarity to how PC1 and PC2 represent the largest variation in the data. I used that explanation in my written analysis.
- **Debugging: Pixel Intensity Histograms:**
 - **Purpose:** While building the histogram using Matplotlib `pyplot.hist` (https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.html), I faced an issue where the histogram bars were not reflecting the correct intensity ranges.
 - **Impact:** AI explained that the `bins` parameter needed to be explicitly set to 256 for pixel data. I tested this fix, understood why it worked, and applied it.
- **Coding: Beautification of graphs and asking about best options.**
- **Editing: Written Analysis (Phase 0):**
 - **Purpose:** After writing my observations for all five figures, I passed the text through AI to check for clarity and grammatical correction.
 - **Impact:** Only small language fixes were applied while the ideas and analysis stayed entirely mine.

PHASE 1: Hypothesis and Model Justification

Tool: Claude / ChatGPT

- **Ideation: Model Selection for Fashion-MNIST Dataset:**

- **Purpose:** After doing my own research on common machine learning models used for image processing and why they are used, I shortlisted a few that seemed suitable for the Fashion-MNIST dataset. I then wrote my understanding based on Phase 0 and shared it with Claude to check whether my reasoning was correct and to clarify any mistakes before finalising the model choices.
- **Impact:** AI explained that flat models like Logistic Regression would struggle due to loss of spatial structure, SVM would perform moderately with kernel tricks but still suffer from flattening, and CNN would perform best by preserving the 2D layout. This confirmed my own Phase 0 observations and helped me justify my final model selection in the report.

- **Editing: Hypothesis Write-up:**

- **Purpose:** My hypothesis was formed entirely from my Phase 0 observations. After writing it up, I used AI to check whether my argument about spatial features and CNNs was being communicated clearly. Sources consulted before writing included CS231n Convolutional Networks (<https://cs231n.github.io/convolutional-networks/>), CS231n Image Classification (<https://cs231n.github.io/classification/>), and the Scikit-learn SVM Guide (<https://scikit-learn.org/stable/modules/svm.html>).
- **Impact:** AI suggested minor restructuring in one paragraph. I rewrote it myself using that feedback as a guide.

PHASE 2: CNN Implementation

Tool: Claude / ChatGPT

- **Debugging: Softmax Numerical Overflow:**

- **Purpose:** During early testing, my softmax function failed because `np.exp(z)` gave `inf` for very large scores, showing (`inf / inf = NaN`). Because of this, the model just output `nan` for every prediction. The thing which needs to be done was to ask the AI how to fix this overflow without changing the real math result.
- **Impact:** The AI explained a standard stability trick. From this observation, one important thing is that subtracting the largest number in the row before using `exp()` keeps all values ≤ 0 . I tested this fix, understood why it worked, and applied it.

- **Understanding: CNN Concepts (ReLU, Dropout, Convolution):**

- **Purpose:** I used AI along with CS231n notes (<https://cs231n.github.io/neural-networks-1/>, <https://cs231n.github.io/convolutional-networks/>) to understand how ReLU, dropout, and convolution work mathematically before implementing them from scratch.
- **Impact:** The explanation helped me correctly implement the forward and backward passes for each component. All code was written by me after understanding the concepts.

- **Understanding: Results Interpretation:**

- **Purpose:** After training was complete, I asked AI how to correctly read a confusion matrix and a training loss curve.
- **Impact:** It helped me link the class confusion patterns to the Phase 0 PCA and similarity observations. All written analysis and conclusions were my own.

- **Editing: Implementation Write-up:**

- **Purpose:** After writing my implementation decisions for the CNN, I used AI to proof-read for clarity and check any grammatical errors.
- **Impact:** AI suggested some changes. I then rewrote it in my own words.

PHASE 3: Reflection

Tool: Claude / ChatGPT

- **Clarification: Interpreting Misleading Observations:**

- **Purpose:** After writing my initial answer for Question 3 about misleading observations, I asked AI to check whether my reasoning about pixel intensity histograms being partially misleading was technically sound.
- **Impact:** AI confirmed the reasoning was correct and suggested being more specific about why class-wise averages still carry limited useful information even though global averages do not. I rewrote the paragraph myself using that clarification.

- **Ideation: Future Approach (Question 4):**

- **Purpose:** After writing my own ideas about what I would do differently, I asked AI whether HOG features and data augmentation were realistic improvements for Fashion-MNIST specifically, to confirm I was not suggesting something technically incorrect.
- **Impact:** AI confirmed both approaches are standard and well-supported for this dataset. All final written content and justifications were my own, rooted in the Phase 0 figures.

- **Editing: Reflection Write-up:**

- **Purpose:** After completing both reflection answers, I passed the text through AI to check for clarity and grammatical errors.
- **Impact:** Only minor language corrections were applied. All observations, reasoning, and conclusions remained entirely my own.

AI Usage Disclosure [Navaal Noshi (2023-EE-5)]

PHASE 0 :Figures and Analysis

Tool: Claude

- **Clarification:Pixel Importance Map:**

- **Purpose:** After reading the Scikit-learn decomposition documentation (<https://scikit-learn.org/stable/modules/decomposition.html>), I asked AI to confirm whether my understanding of class mean subtraction was correct before coding it.
- **Impact:** AI helped by explaining my own understanding. The basic idea for this is what it added a lot more clarity to my whole concept.

- **Clarification: Visual Similarity Heatmap:**

- **Purpose:** After reading the NumPy `corrcoef` (<https://numpy.org/doc/stable/reference/generated/numpy.corrcoef.html>), I asked AI to clarify what the output correlation values meant visually in a heatmap context.
- **Impact:** The explanation helped me label and interpret the heatmap correctly.

- **Debugging: Pixel Frequency Distribution:**

- **Purpose:** While building the histogram using Matplotlib `pyplot.hist` (https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.html), I faced an issue where the histogram bars were not reflecting the correct intensity ranges. Took help to understand the problem.
- **Impact:** AI explained that the `bins` parameter needed to be explicitly set to 256 for pixel data. I tested this fix, understood why it worked, and applied it.

- **Coding: Beautification of Graphs and asking about best options**

- **Understanding and Editing: Written Analysis (Phase 0):**

- **Purpose:** After writing my observations for all five figures, I passed the text through AI to check for clarity and grammatical correction .
- **Impact:** AI suggestions were added to the report.

PHASE 1:Hypothesis and Model Justification

Tool:Claude

- **Ideation: Model Selection for Fashion-MNIST Dataset:**

- **Purpose:** As when this phase was assigned we haven't covered many models in the class. So, after doing my research and before finalizing my model choices, I asked AI which models would work well on the Fashion-MNIST dataset and which would not, along with the reasons behind each. This included understanding why certain models are limited for image data.
- **Impact:** AI explained that flat models like Logistic Regression would struggle due to loss of spatial structure, SVM would perform moderately with kernel tricks but still suffer from flattening, and CNN would perform best by preserving the 2D layout. This confirmed my own Phase 0 observations and helped me justify my final model selection in the report.

- **Editing:Hypothesis Write-up:**

- **Purpose:** My hypothesis was formed entirely from my Phase 0 observations. After writing it up, I used AI to check whether about CNN spatial preservation versus SVM flattening was being communicated clearly. Sources consulted before writing included CS231n Convolutional Networks (<https://cs231n.github.io/convolutional-networks/>), CS231n Image Classification (<https://cs231n.github.io/classification/>),.
- **Impact:** AI suggested restructuring one paragraph, I rewrote it myself using that feedback as a guide.

PHASE 2: Model Implementation

Tool: Claude

- **Debugging:Softmax Numerical Overflow:**

- **Purpose:** During early testing, my softmax function failed because `np.exp(z)` gave `inf` for very large scores. It was showing (`inf / inf = NaN`). Because of this, the model just output `nan` for every prediction. The thing which needs to be done was to ask the AI how to fix this overflow without changing the real math result.
- **Impact:** The AI explained a standard stability trick. From this observation, one important thing is that subtracting the largest number in the row before using `exp()` keeps all values ≤ 0 .

- **Debugging: Logistic Regression (NumPy from scratch):**

- **Purpose:** When I was writing the math for softmax and cross-entropy gradient using the CS231n guide (<https://cs231n.github.io/linear-classify/>), I got a shape mismatch error while updating the weights. The basic idea for this is what the sizes of my matrices just did not match up. AI explained me what exactly dimension error is.
- **Impact:** The AI explained that the gradient matrix needed to be flipped (transposed) before we multiply them together. From this observation, one important thing is that it makes the shapes match perfectly so the math can work.

- **Debugging: SVM (Scikit-learn):**

- **Purpose:** When setting up the One-vs-Rest SVM strategy, based on Scikit-learn Multi-class documentation (<https://scikit-learn.org/stable/modules/multiclass.html>), I got a parameter conflict error between `decision_function_shape` and my cross-validation setup. I asked AI to explain the error message.
- **Impact:** AI helped me to fix this bug. The basic idea for this is what the model could not read it. The thing which needs to be done was to delete it. Because of this, I removed the setting after checking the guide (<https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>).

- **Editing: Implementation Write-up:**

- **Purpose:** After writing up my implementation decisions for both models, I used AI to proofread for clarity and check any grammatical error. Also to make comments better in code.
- **Impact:** AI suggested some changes. Then I rewrote it in my own words.

PHASE 3: Reflection

Tool: Claude

- **Editing: Reflection Write-up:**

- **Purpose:** After writing my reflection answers based on my actual results and Phase 0 observations, I used AI to check for clarity and grammatical correctness.
- **Impact:** AI suggested some language improvements. I rewrote everything in my own words using that feedback as a guide.